4190.428 001 Introduction to Machine Learning

Gradient Descent

Jaesik Park
CSE & IPAI, SNU
jaesik.park@snu.ac.kr

[Recall] Neural Networks

- Neural network training aims to update the parameters in a direction that reduces the loss.

- Equation:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

- **Back-propagation**: An algorithm for computing these gradient efficiently .

Modulation

- A neural network can be viewed as a composition of small modules

$$J = M_k(M_{k-1}(\cdots M_1(x)))$$

- M_i : linear layer, activation function, loss function
- Some modules have parameters
  - linear layer (W, b)
- Some modules have no parameters
  - ReLU, sigmoid

Forward pass

- We compute values from the input to the loss and store intermediate variables.

$$u^{[0]} = x$$
$$u^{[i]} = M_i(u^{[i-1]})$$
$$u^{[k]} = J$$

- Store intermediate variables $u^{[i]}$
- These stored values are needed later to compute local derivatives during backward pass.

$$x = u^{[0]} \to u^{[1]} \to u^{[2]} \to \cdots \to u^{[k]} = J$$

Chain rule

- Back-propagation repeatedly applies the chain rule from the end of the network to the beginning.
- Simple example

  $z \in \mathbb{R}^m$, $u \in \mathbb{R}^n$, J: scalar

$$z \xrightarrow{g} u \xrightarrow{f} J$$

$$u = g(z), \qquad J = f(u)$$

$$\frac{\partial J}{\partial z_i} = \sum_{j=1}^{n} \frac{\partial J}{\partial u_j} \frac{\partial u_j}{\partial z_i}$$

Backward function

- Each module has a backward function that converts an upstream gradient into a downstream gradient.

$$z \xrightarrow{g} u \xrightarrow{f} J$$

- With given upstream gradient $\left(\frac{\partial J}{\partial u}\right) = v$,

$$\frac{\partial J}{\partial z} = \mathscr{B}[g, z](v)$$

Backward function - linear layer

- For a linear layer, input gradients and parameter gradients can be computed using simple matrix multiplication ($MM_{W,b}(z) = Wz + b$)
- Forward:

$$u = Wz + b$$

- Backward
  - Given upstream gradient $\left(\frac{\partial J}{\partial u}\right) = v$,

$$\mathscr{B}[MM, z](v) = W^{\top} v$$

- Gradient used for parameter updates

$$\mathscr{B}[MM, W](v) = vz^{\top}, \quad \mathscr{B}[MM, b](v) = v$$

Backward function - Activation & Losses

- Activation function $\sigma(z)$: element-wise activation function

$$\mathscr{B}[\sigma, z](v) = \sigma'(z) \odot v$$

($\odot$: element wise multiplication)

  - e.g. ReLU:

$$\mathrm{ReLU}(z) = \max(0,z)$$

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$, we already calculated z during forward pass.

- Loss functions
  - Squared loss: $l_{MSE} \triangleq \frac{1}{2}(z-y)^2$, $\mathscr{B}[l_{MSE}, z](v) = (z-y) \cdot v$
  - Logistic loss: $\ell_{\text{logistic}}(t, y) \triangleq y \log(1 + \exp(-t)) + (1-y)\log(1 + \exp(t))$.

$$\mathscr{B}[\ell_{\text{logistic}}, t](v) = \frac{\partial \ell_{\text{logistic}}(t, y)}{\partial t} \cdot v = \left(\frac{1}{1 + \exp(-t)} - y \right) \cdot v$$

Back-propagation for MLP

- Back-propagation connects local backward rules from the last layer to the first layer to compute all parameter gradients.
- MLP with logistic loss

$$
\begin{aligned}
z^{[1]} &= \mathrm{MM}_{W^{[1]},b^{[1]}}(x), \\
a^{[1]} &= \sigma(z^{[1]}), \\
z^{[2]} &= \mathrm{MM}_{W^{[2]},b^{[2]}}(a^{[1]}), \\
a^{[2]} &= \sigma(z^{[2]}), \\
&\vdots \\
z^{[r]} &= \mathrm{MM}_{W^{[r]},b^{[r]}}(a^{[r-1]}), \\
J &= \ell_{\text{logistic}}(z^{[r]}, y)\,.
\end{aligned}
$$

Back-propagation for MLP

- Backward
  - First, we have that

$$\frac{\partial J}{\partial z^{[r]}} = \mathscr{B}[\ell_{\text{logistic}}, z^{[r]}]\left(\frac{\partial J}{\partial J}\right) = \mathscr{B}[\ell_{\text{logistic}}, z^{[r]}](1)\,.$$

  - Then we iteratively compute

$$\begin{aligned}
\frac{\partial J}{\partial a^{[r-1]}} &= \mathscr{B}[\mathrm{MM}, a^{[r-1]}]\left(\frac{\partial J}{\partial z^{[r]}}\right), \\
\frac{\partial J}{\partial z^{[r-1]}} &= \mathscr{B}[\sigma, z^{[r-1]}]\left(\frac{\partial J}{\partial a^{[r-1]}}\right), \\
&\vdots \\
\frac{\partial J}{\partial z^{[1]}} &= \mathscr{B}[\sigma, z^{[1]}]\left(\frac{\partial J}{\partial a^{[1]}}\right).
\end{aligned}$$

Back-propagation for MLP

- Backward
  - Finally we compute gradient of parameters

$$
\begin{aligned}
\frac{\partial J}{\partial W^{[r]}} &= \mathscr{B}[\mathrm{MM}, W^{[r]}]\left(\frac{\partial J}{\partial z^{[r]}}\right), \\
\frac{\partial J}{\partial b^{[r]}} &= \mathscr{B}[\mathrm{MM}, b^{[r]}]\left(\frac{\partial J}{\partial z^{[r]}}\right), \\
&\vdots \\
\frac{\partial J}{\partial W^{[1]}} &= \mathscr{B}[\mathrm{MM}, W^{[1]}]\left(\frac{\partial J}{\partial z^{[1]}}\right), \\
\frac{\partial J}{\partial b^{[1]}} &= \mathscr{B}[\mathrm{MM}, b^{[1]}]\left(\frac{\partial J}{\partial z^{[1]}}\right).
\end{aligned}
$$

Back-propagation for MLP

- Algorithm of Back-propagation

Algorithm 3 Back-propagation for multi-layer neural networks.

1: **Forward pass.** Compute and store the values of $a^{[k]}$'s, $z^{[k]}$'s, and J using the equations (7.72).

2: **Backward pass.** Compute the gradient of loss J with respect to $z^{[r]}$:

$$\frac{\partial J}{\partial z^{[r]}} = \mathcal{B}[\ell_{\text{logistic}}, z^{[r]}](1) = \left(1/(1+\exp(-z^{[r]})) - y\right) . \tag{7.76}$$

3: **for** $k = r-1$ to 0 **do**

4: Compute the gradient with respect to parameters $W^{[k+1]}$ and $b^{[k+1]}$.

$$\begin{aligned} \frac{\partial J}{\partial W^{[k+1]}} &= \mathcal{B}[\text{MM}, W^{[k+1]}]\left(\frac{\partial J}{\partial z^{[k+1]}}\right) \\ &= \frac{\partial J}{\partial z^{[k+1]}} a^{[k]\top} . \end{aligned} \tag{7.77}$$

$$\begin{aligned} \frac{\partial J}{\partial b^{[k+1]}} &= \mathcal{B}[\text{MM}, b^{[k+1]}]\left(\frac{\partial J}{\partial z^{[k+1]}}\right) \\ &= \frac{\partial J}{\partial z^{[k+1]}} . \end{aligned} \tag{7.78}$$

Back-propagation for MLP

- Algorithm of Back-propagation

Algorithm 3 Back-propagation for multi-layer neural networks.

5: When $k \geq 1$, compute the gradient with respect to $z^{[k]}$ and $a^{[k]}$.

$$\begin{aligned} \frac{\partial J}{\partial a^{[k]}} &= \mathcal{B}[\sigma, a^{[k]}] \left(\frac{\partial J}{\partial z^{[k+1]}} \right) \\ &= W^{[k+1]\top} \frac{\partial J}{\partial z^{[k+1]}} . \end{aligned} \tag{7.79}$$

$$\begin{aligned} \frac{\partial J}{\partial z^{[k]}} &= \mathcal{B}[\sigma, z^{[k]}] \left(\frac{\partial J}{\partial a^{[k]}} \right) \\ &= \sigma'(z^{[k]}) \odot \frac{\partial J}{\partial a^{[k]}} . \end{aligned} \tag{7.80}$$

QnA

- Any questions?
- Note: this lecture notes are largely adopted from
  - MIT 6.036(2019 Fall), Stanford CS229(2023-2024 Summer)